

REFERENCIA RÁPIDA – TIPOS DE JOIN

Tipo	Devuelve	Cuándo usar
INNER JOIN	Solo filas con coincidencia en ambas tablas	Listar pedidos con datos de usuario
LEFT JOIN	Todas las filas de la izquierda + coincidencias	Detectar usuarios sin pedidos
RIGHT JOIN	Todas las filas de la derecha + coincidencias	Detectar productos sin ventas
FULL OUTER JOIN	Todas las filas de ambas tablas	Vista completa aunque no haya relación

FUNCIONES DE AGREGACIÓN

COUNT(*) número de filas	SUM(col) suma de valores
AVG(col) media de valores	MAX(col) valor máximo
MIN(col) valor mínimo	GROUP BY col agrupar filas por valor
HAVING cond filtrar grupos (como WHERE pero tras GROUP BY)	ORDER BY col DESC ordenar de mayor a menor

TABLAS DEL BLOQUE – ESTRUCTURA COMPLETA

```
CREATE TABLE usuarios (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE NOT NULL,  
  edad INT,  
  activo BOOLEAN DEFAULT TRUE  
);  
CREATE TABLE productos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  precio DECIMAL(10,2) NOT NULL,  
  stock INT NOT NULL,  
  categoria VARCHAR(50),  
  activo BOOLEAN DEFAULT TRUE  
);  
CREATE TABLE pedidos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  usuario_id INT,  
  producto_id INT,  
  cantidad INT NOT NULL,  
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id),  
  FOREIGN KEY (producto_id) REFERENCES productos(id)  
);
```

✗ ERRORES COMUNES

- Olvidar los alias (**u.**, **pe.**) en JOINS con columnas del mismo nombre → ambigüedad de columna
- Insertar en **pedidos** con un **usuario_id** inexistente → violación de clave foránea
- Usar **WHERE** en vez de **HAVING** para filtrar grupos agregados → error de sintaxis

🔗 GROUP BY + JOIN COMBINADOS

```
TOTAL VENDIDO POR PRODUCTO  
  
SELECT p.nombre,  
       SUM(pe.cantidad) AS total_vendido  
FROM   pedidos pe  
JOIN   productos p ON pe.producto_id = p.id  
GROUP BY p.nombre  
ORDER BY total_vendido DESC;
```

NÚMERO DE PEDIDOS POR USUARIO

```
SELECT u.nombre,  
       COUNT(*) AS num_pedidos  
FROM   pedidos pe  
JOIN   usuarios u ON pe.usuario_id = u.id  
GROUP BY u.nombre  
ORDER BY num_pedidos DESC;
```

PRODUCTOS SIN VENTAS – LEFT JOIN + IS NULL

```
SELECT p.nombre  
FROM   productos p  
LEFT JOIN pedidos pe  
       ON p.id = pe.producto_id  
WHERE  pe.id IS NULL; -- no tiene pedidos
```

FILTRAR CON WHERE DESPUÉS DE JOIN

```
SELECT u.nombre, pe.cantidad  
FROM   pedidos pe  
JOIN   usuarios u ON pe.usuario_id = u.id  
WHERE  u.activo = true;
```

SUBCONSULTA – PRODUCTO MÁS CARO COMPRADO

```
SELECT *  
FROM   pedidos  
WHERE  producto_id = (  
  SELECT id  
  FROM   productos  
  ORDER BY precio DESC  
  LIMIT 1  
);
```

📌 HAVING VS WHERE

	WHERE	HAVING
Filtra	Filas individuales	Grupos ya agregados
Se aplica	Antes del GROUP BY	Después del GROUP BY
Puede usar	Columnas normales	Funciones agregadas (SUM, COUNT...)

```
-- Usuarios con más de 3 pedidos  
SELECT u.nombre, COUNT(*) AS total  
FROM   pedidos pe  
JOIN   usuarios u ON pe.usuario_id = u.id  
GROUP BY u.nombre  
HAVING COUNT(*) > 3;
```

Relaciones y JOIN

BLOQUE 5 · CHULETARIO · TRÍPTICO A4

MODELO RELACIONAL – 3 TABLAS



TIPOS DE JOIN DE UN VISTAZO

INNER JOIN Solo filas con coincidencia en ambas tablas. Las no coincidentes se descartan.	LEFT JOIN Todas las filas de la tabla izquierda. Si no hay coincidencia → NULL en la derecha.
---	---

TIPOS DE RELACIONES

Relación	Ejemplo	Implementación
1:N	Usuario → muchos pedidos	FK en la tabla "N"
N:1	Muchos pedidos → 1 producto	FK en la tabla "N"
N:M	Alumnos ↔ Asignaturas	Tabla intermedia con 2 FK

CONTENIDOS DE ESTE TRÍPTICO

- Interior izq. — PK, FK, relaciones, INNER JOIN completo
- Interior cent. — LEFT JOIN, ORDER BY, GROUP BY con ejemplos
- Interior der. — JOINS en JDBC con PreparedStatement, patrones
- Solapa — GROUP BY + JOIN combinados, WHERE vs HAVING, subconsultas
- Contraportada — Tabla de JOINS, funciones de agregación, tablas del bloque

🔑 CLAVE PRIMARIA (PRIMARY KEY)

- Identifica de forma **única** cada fila de una tabla
- No puede ser `NULL` ni repetirse
- Normalmente `INT AUTO_INCREMENT`

```
id INT AUTO_INCREMENT PRIMARY KEY
```

🔗 CLAVE FORÁNEA (FOREIGN KEY)

- Relaciona una tabla con otra usando su PK
- Garantiza **integridad referencial**: no puedes insertar un valor que no exista en la tabla referenciada

```
usuario_id INT,
FOREIGN KEY (usuario_id)
  REFERENCES usuarios(id)
-- Cada pedido pertenece a un usuario existente
```

Si insertas en `pedidos` con un `usuario_id` que no existe en `usuarios` → error: *Cannot add or update a child row: a foreign key constraint fails*

🔗 INNER JOIN – SOLO COINCIDENCIAS

Devuelve solo las filas que tienen correspondencia en **ambas** tablas. Si un pedido no tiene usuario (o viceversa), no aparece.

JOIN ENTRE DOS TABLAS

```
SELECT u.nombre, pe.cantidad, pe.fecha
FROM pedidos pe
JOIN usuarios u
  ON pe.usuario_id = u.id;
```

JOIN ENTRE TRES TABLAS

```
SELECT u.nombre,
       p.nombre AS producto,
       pe.cantidad
FROM pedidos pe
JOIN usuarios u ON pe.usuario_id = u.id
JOIN productos p ON pe.producto_id = p.id;
```

Elemento	Para qué sirve
Alias (<code>pe</code> , <code>u</code> , <code>p</code>)	Abreviar nombres de tabla y evitar ambigüedades
<code>ON t1.fk = t2.pk</code>	Condición de relación entre tablas
<code>AS alias_col</code>	Renombrar columna en el resultado

⚙️ ORDER BY – ORDENAR RESULTADOS

```
SELECT * FROM productos
  ORDER BY precio DESC; -- de mayor a menor

SELECT * FROM usuarios
  ORDER BY nombre ASC;  -- alfabético
```

🔗 LEFT JOIN – TODO LO DE LA IZQUIERDA

Devuelve **todas** las filas de la tabla izquierda, aunque no tengan correspondencia en la derecha. Las columnas sin correspondencia aparecen como `NULL`.

TODOS LOS PRODUCTOS, TENGAN PEDIDOS O NO

```
SELECT p.nombre, pe.id AS pedido_id
FROM productos p
LEFT JOIN pedidos pe
  ON p.id = pe.producto_id;
-- Si p.id no está en pedidos + pedido_id = NULL
```

DETECTAR PRODUCTOS SIN VENTAS

```
SELECT p.nombre
FROM productos p
LEFT JOIN pedidos pe
  ON p.id = pe.producto_id
WHERE pe.id IS NULL;
-- Solo los que NO tienen pedidos
```

📊 GROUP BY – AGRUPAR Y AGREGAR

`GROUP BY` agrupa las filas con el mismo valor en una columna. Se usa junto con funciones de agregación (`SUM`, `COUNT`, `AVG`...).

TOTAL VENDIDO POR PRODUCTO

```
SELECT producto_id,
       SUM(cantidad) AS total
FROM pedidos
GROUP BY producto_id;
```

PEDIDOS POR USUARIO (CON NOMBRE)

```
SELECT u.nombre, COUNT(*) AS pedidos
FROM pedidos pe
JOIN usuarios u ON pe.usuario_id = u.id
GROUP BY u.nombre
ORDER BY pedidos DESC; -- ranking
```

PRODUCTOS CON MÁS DE 10 VENTAS (HAVING)

```
SELECT p.nombre, SUM(pe.cantidad) AS total
FROM pedidos pe
JOIN productos p ON pe.producto_id = p.id
GROUP BY p.nombre
HAVING SUM(pe.cantidad) > 10;
```

🔗 JOIN EN JDBC CON PREPAREDSTATEMENT

LISTAR PEDIDOS DE UN USUARIO POR NOMBRE

```
String sql =
    "SELECT u.nombre, p.nombre AS producto, pe.cantidad"
    + " FROM pedidos pe"
    + " JOIN usuarios u ON pe.usuario_id = u.id"
    + " JOIN productos p ON pe.producto_id = p.id"
    + " WHERE u.id = ?";

try {
    Connection conn = DriverManager.getConnection(url, usr, pwd);
    PreparedStatement ps = conn.prepareStatement(sql);
} {
    ps.setInt(1, usuarioId);
    ResultSet rs = ps.executeQuery();
    while (rs.next()) {
        System.out.printf("%-12s %-15s %d\n",
            rs.getString("nombre"),
            rs.getString("producto"),
            rs.getInt ("cantidad"));
    }
} catch (SQLException e) { e.printStackTrace(); }
```

INSERTAR PEDIDO + REDUCIR STOCK (DOS QUERIES)

```
// 1. Insertar pedido
String ins = "INSERT INTO pedidos (usuario_id, producto_id, cantidad)"
    + " VALUES (?, ?, ?)";
PreparedStatement psIns = conn.prepareStatement(ins);
psIns.setInt(1, usuId); psIns.setInt(2, prodId); psIns.setInt(3, cant);
psIns.executeUpdate();

// 2. Reducir stock
String upd = "UPDATE productos SET stock = stock - ? WHERE id = ?";
PreparedStatement psUpd = conn.prepareStatement(upd);
psUpd.setInt(1, cant); psUpd.setInt(2, prodId);
psUpd.executeUpdate();
```

📄 TABLA RESUMEN DEL BLOQUE

Consulta	Patrón SQL
Datos cruzados	<code>JOIN ... ON t1.fk = t2.pk</code>
Incluir sin relación	<code>LEFT JOIN ... WHERE t2.id IS NULL</code>
Total por grupo	<code>GROUP BY col + SUM()</code>
Filtrar grupos	<code>HAVING SUM() > n</code>
Ranking	<code>ORDER BY total DESC</code>
Elemento extremo	<code>ORDER BY ... LIMIT 1</code>
Subconsulta	<code>WHERE col = (SELECT ...)</code>

Regla de oro: usa alias cortos (`u`, `p`, `pe`) para evitar ambigüedades · empieza el `FROM` por la tabla "central" (la que tiene las FK) · `INNER JOIN` para listados, `LEFT JOIN` para detectar ausencias.